

CPSC 340: Machine Learning and Data Mining

Convolutions

Admin

- HW6 is due Wed Dec 3
- Grad students: project reports due Mon Dec 15
- Final: Thu Dec 18 (8:30am)

Last Time: Automatic Differentiation (AD)

- Automatic differentiation (AD):
 - Takes code that computes a function.
 - Produces code that computes derivatives.
 - No error due to numerical approximations but no formula (computed algorithmically).
- AD writes functions as a **sequence of simple compositions**:
 - $f_5(f_4(f_3(f_2(f_1(x))))))$
- AD writes derivatives using **chain rule**:
 - $f'(x) = f_5'(f_4(f_3(f_2(f_1(x)))))*f_4'(f_3(f_2(f_1(x))))*f_3'(f_2(f_1(x)))*f_2'(f_1(x))*f_1'(x).$
- We showed “**forward pass**” and “**backward pass**” for single-variable AD:
 - $\alpha_1 = f_1(x), \alpha_2 = f_2(\alpha_1), \alpha_3 = f_3(\alpha_2), \alpha_4 = f_4(\alpha_3), \alpha_5 = f_5(\alpha_4) = f(x).$
 - $\beta_5 = 1*f_5'(\alpha_4), \beta_4 = \beta_5*f_4'(\alpha_3), \beta_3 = \beta_4*f_3'(\alpha_2), \beta_2 = \beta_3*f_2'(\alpha_1), \beta_1 = \beta_2*f_1'(x) = f'(x).$
- Cost of **computing $f'(x)$** is same cost as of computing **$f(x)$** .  that is interesting!

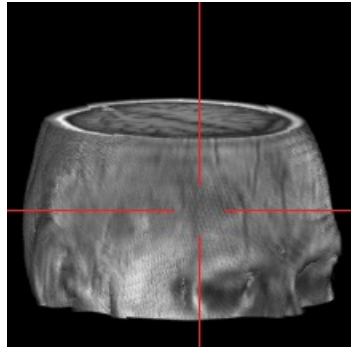
Next Topic: Convolutions



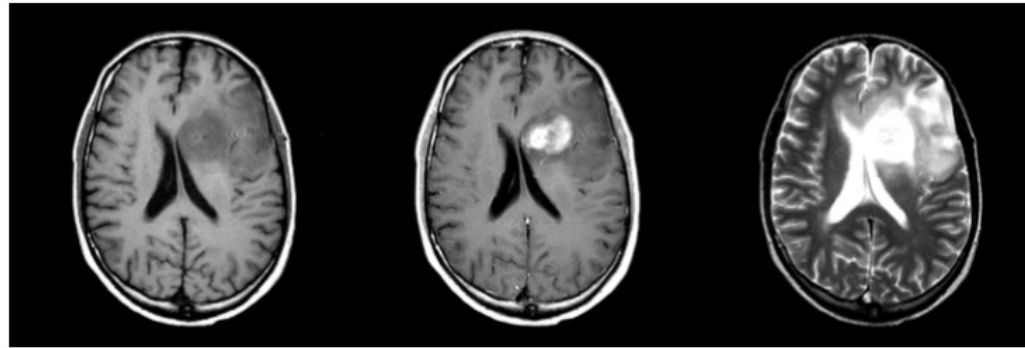
still linear on non-linear features

Motivation: Automatic Brain Tumor Segmentation

- Task: labeling tumors and normal tissue in multi-modal MRI data.



Input:



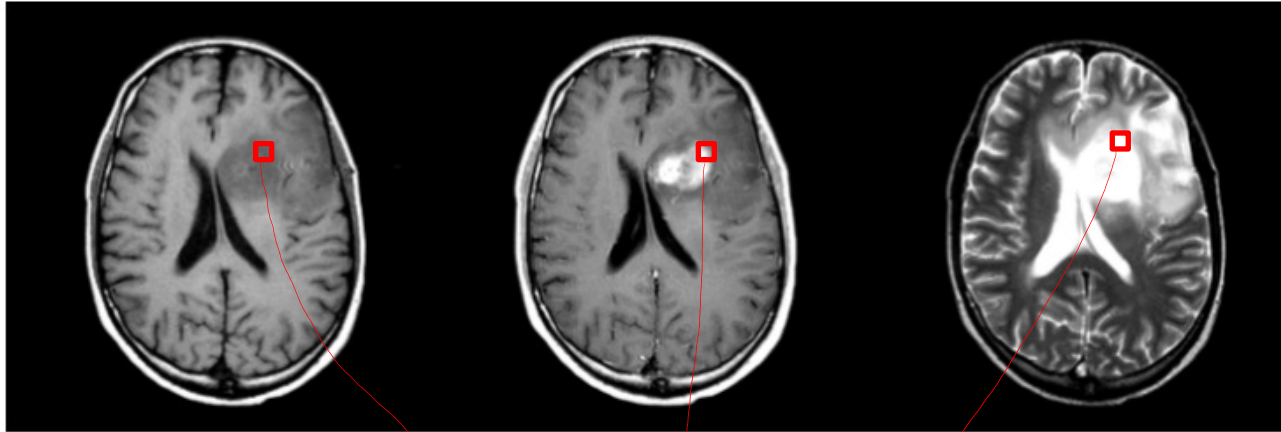
Output:



- Applications:
 - Radiation therapy target planning, quantifying treatment responses.
 - Mining growth patterns, image-guided surgery.
- Challenges:
 - Variety of tumor appearances, similarity to normal tissue.
 - Grumbly scientist in 2003: “you are never going to solve this problem.”

Naïve Voxel-Level Classifier

- We could treat classifying a voxel as **supervised learning**:
 - Standard representation of image: each pixel gets “intensity” between 0 and 255.



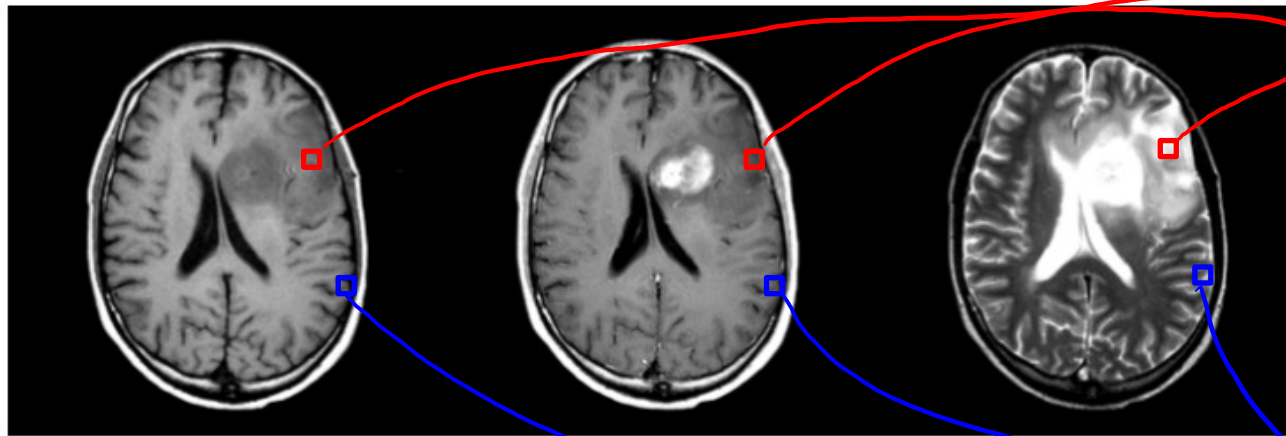
$$x_i = (98, 187, 246)$$

$$y_i = \text{"tumour"}$$

- We can formulate predicting y_i given x_i as supervised learning.
- But it **does not work** at all with these features.

Need to Summarize Local Context

- The individual pixel intensity values are almost meaningless:
 - The same x_i could lead to different y_i .



$x_i = (87, 54, 173)$

$y_i = \text{"tumour"}$

$x_i = (87, 54, 173)$

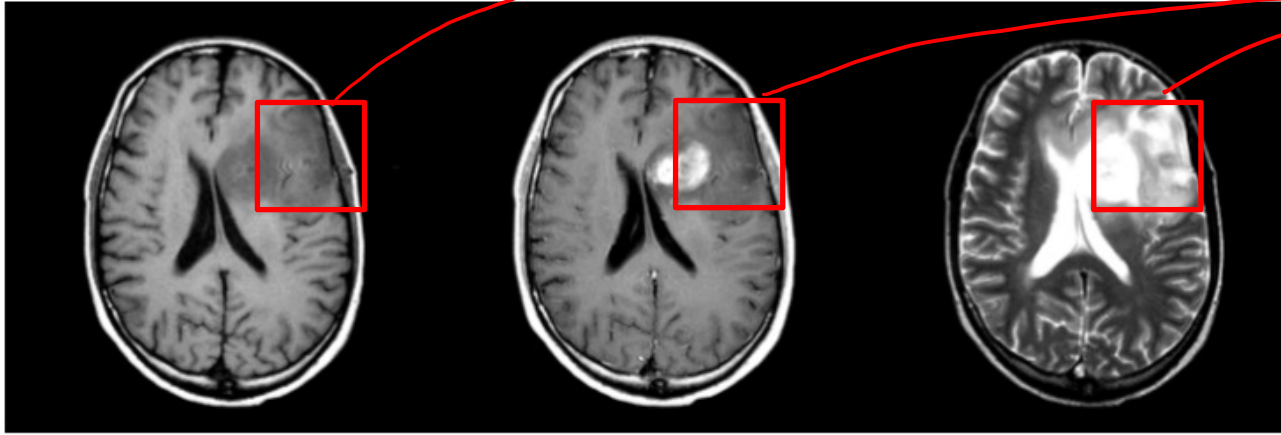
$y_i = \text{"normal"}$

Same input
but different
labels.

- Intensities not standardized.
- Non-trivial overlap in signal for different tissue types.
- "Partial volume" effects at boundaries of tissue types.

Need to Summarize Local Context

- We need to **represent the “context”** of the pixel (what is around it).



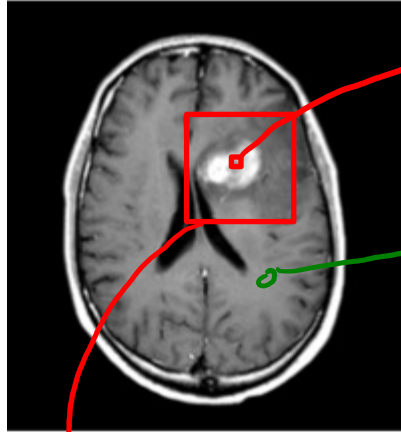
$x_i = (\text{---} , \text{---} , \text{---})$

- Include all the values of **neighbouring pixels** as extra features?
 - Run into coupon collection problems: **requires lots of data** to find patterns.
- Measure neighbourhood **summary statistics** (mean, variance, histogram)?
 - Variation on bag of words problem: **loses spatial information** present in voxels.
- Standard approach uses **convolutions** to represent neighbourhood. $\hookrightarrow$ 3D Pixels
loses how pixels are close to each other

↳ we want so

Other

Example: Measuring “brightness” of an Area



- This pixel is in a “bright” area of the image, which reflects “bleeding” of tumour.
- But the actual numeric intensity value of the pixel is the **same as in darker** “gray matter” areas.

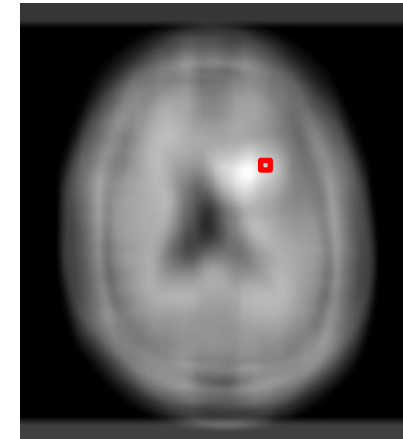
- I want a feature saying “this pixel is in a bright area of the image”.
- This will help us identify that it’s a tumour pixel.

Some thing
more than mean
var
...

- Obvious way to measure brightness in area: take **average pixel intensity** in “neighbourhood”.

$$z = \frac{1}{|nei|} \sum_{k \in nei} x_k \quad \left. \vphantom{\sum_{k \in nei} x_k} \right\} \rightarrow \text{new feature is average value in neighbourhood.}$$

- Applying this “averaging” to **every pixel** gives a new image:
- We can use “**pixel value in new image**” as a new feature.
 - New feature helps identify if a pixel is in a “bright” area.



The annoying thing about squares

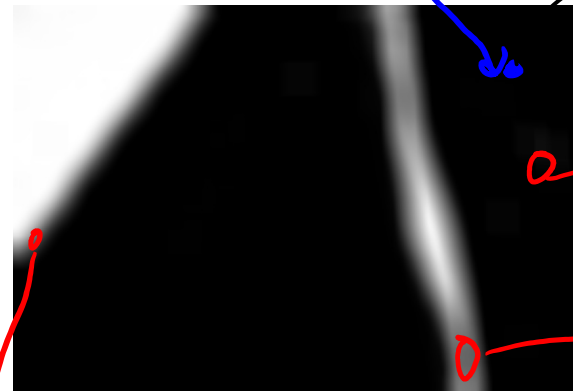
- “Take the average of a square window” **loses spatial information.**

- Example:



Take
average

replace each pixel by the
average value of "window"



Replace Pixel value
by the average.

lots of
spacial
features
of
the image
are excluded

Weird stuff

pixels far away from "edge" become bright.

Noticeable features are "averaged out"

- average is higher, but
location is lost.

"Dark" because line is
"surrounded" by dark areas

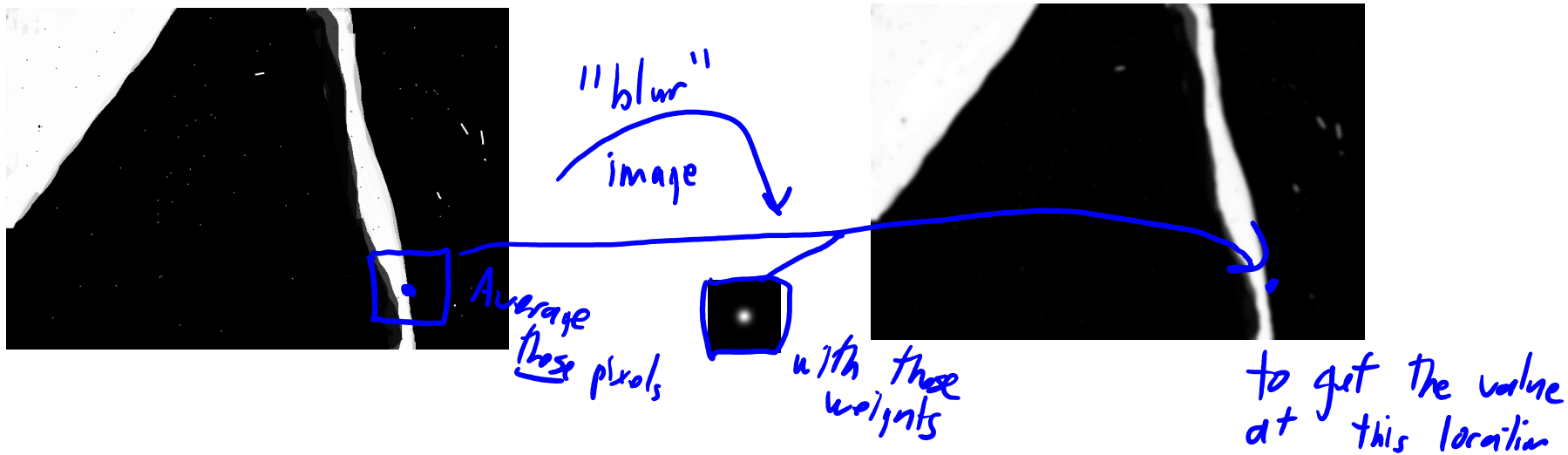
loses a lot!

Fixing the “square” issues

- Consider instead “blurring” the image.

→ Gaussian weights

- Gets rid of “local” noise, but better preserves spatial information.



- How do you “blur”?

- Take **weighted average of window**, putting more “weight” on “close” pixels:

$$z = \sum_{k \in \text{nei}} w_k x_k$$

→ weight on pixel x_k

more weights to closer pixels.

weight are Gaussian

(averaging is special case where all pixels get equal weight)

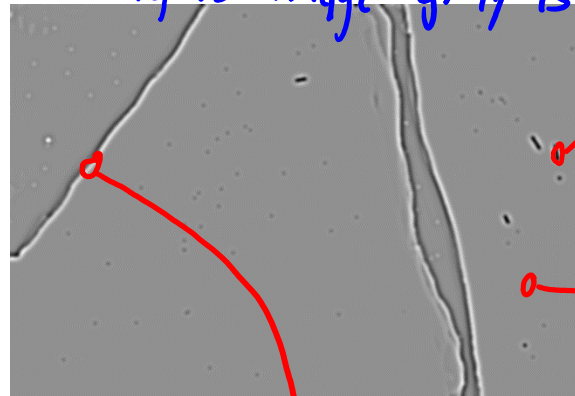
Fixing the “square” issues

- Another neat thing we can do: use **negative weights**.
 - These features can describe **“differences” across space**.

gradient
kind of.



“Average”
with
positive
and
negative
weights



“Signed” image: gray is 0.

“My neighbours are darker than me”

“My neighbours are the same as me”

Useful feature: “My neighbours are brighter than me”

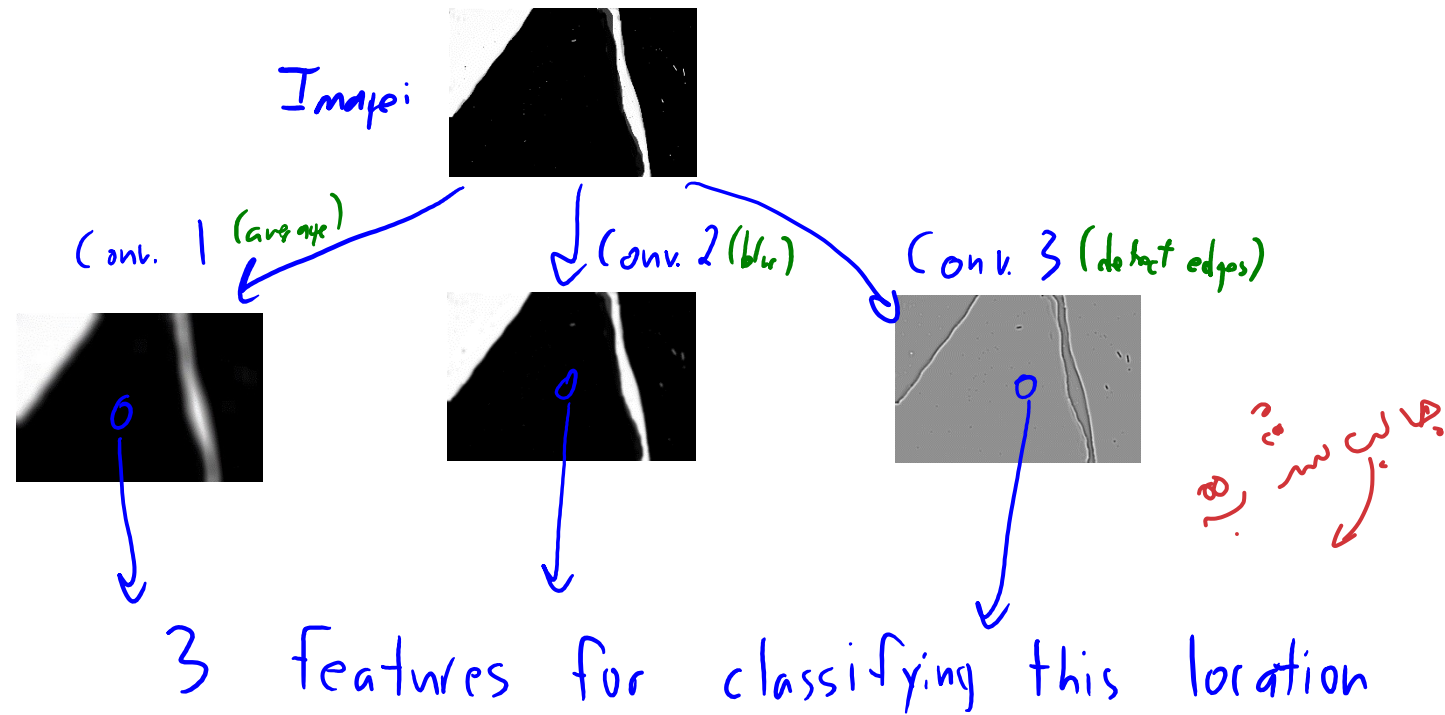
- These “**weighted averages of neighbours**” are called “**convolutions**”.
 - I think of convolutions as the “**words**” that make up image regions.

PCA → it only cares about globally important features

Convolutions: Big Picture

Here (Convolution) → Cares about locally important features.

- How do you use convolutions to get features?
 - Apply **several different convolutions to your image**.
 - Each convolution gives a different “image” value at each location.
 - **Use these different image values to give features** at each location.

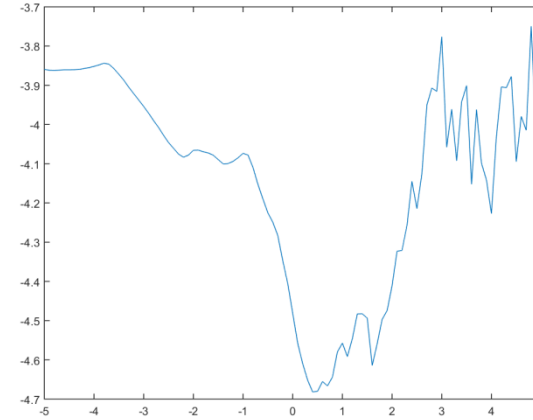


Convolutions: Big Picture

- What can features coming from convolutions represent?
 - Some filters give you an **average value of the neighbourhood**.
 - Some filters **approximate the “first derivative”** in the neighbourhood.
 - “Is there a change from low to dark to bright?”
 - “If so, from which direction in space?”
 - Some filters **approximate the “second derivative”** in the neighbourhood.
 - “Is there a spike or is the change speeding up?”
- Hope: we can characterize **“what happens in a neighbourhood”**,
with just a few numbers.

1D Convolution Example

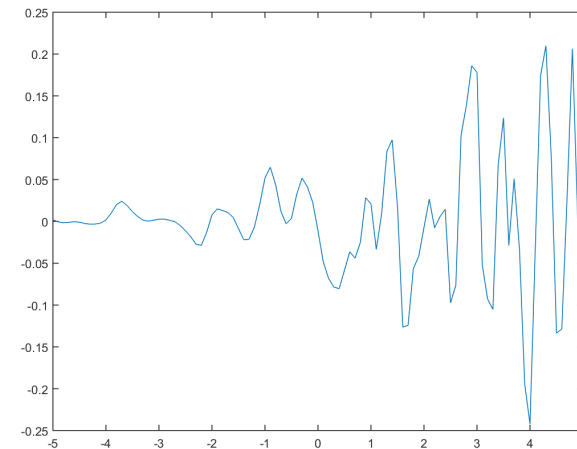
- Consider a 1D “signal” (maybe from sound):
 - We will come back to images later.
- For each “time”:
 - Compute dot-product of signal at surrounding times with a “filter” of weights.



array of weight called filters.

$$w = [-0.1416 \quad -0.1781 \quad -0.2746 \quad 0.1640 \quad 0.8607 \quad 0.1640 \quad -0.2746 \quad -0.1781 \quad -0.1416]$$

- This gives a new “signal”:
 - Measures a property of “neighbourhood”.
 - This particular filter shows a local “how spiky” value.



1D Convolution (notation is specific to this lecture)

- 1D convolution input:

- Signal 'x' which is a vector length 'n'.

- Indexed by $i=1,2,\dots,n$.

$$x = [0 \ 1 \ 1 \ 2 \ 3 \ 5 \ 8 \ 13]$$

- Filter 'w' which is a vector of length '2m+1':

- Indexed by $i=-m,-m+1,\dots,-2,0,1,2,\dots,m-1,m$

$$w = [0 \ -1 \ 2 \ -1 \ 0]$$

$w_{-2} \quad w_{-1} \quad w_0 \quad w_1 \quad w_2$

- Output is a vector of length 'n' with elements:

$$z_i = \sum_{j=-m}^m w_j x_{i+j}$$

- You can think of this as centering w at position 'i',
and taking a dot product of 'w' with that "part" x_i .

1D Convolution

- 1D convolution example:

- Signal 'x': $\rightarrow$ Each is a time step +

0	1	1	2	3	5	8	13
---	---	---	---	---	---	---	----

- Filter 'w':

x	x	x	x	x
0	-1	2	-1	0

- Convolution 'z':

--	--	--	--	--	--	--	--

1D Convolution

- 1D convolution example:

- Signal 'x':

0	1	1	2	3	5	8	13
---	---	---	---	---	---	---	----

- Filter 'w':

0	-1	2	-1	0
---	----	---	----	---

- Convolution 'z':

		-1					
--	--	----	--	--	--	--	--

take dot-product $(0 \cdot 0 + 1 \cdot (-1) + 1 \cdot 2 + 2 \cdot (-1) + 3 \cdot 0)$

1D Convolution

- 1D convolution example:

- Signal 'x':

0	1	1	2	3	5	8	13
---	---	---	---	---	---	---	----

- Filter 'w':

0	-1	2	-1	0
---	----	---	----	---

- Convolution 'z':

		-1	0				
--	--	----	---	--	--	--	--

1D Convolution

- 1D convolution example:

- Signal 'x':

0	1	1	2	3	5	8	13
---	---	---	---	---	---	---	----

- Filter 'w':

0	-1	2	-1	0
---	----	---	----	---

- Convolution 'z':

		-1	0	-1			
--	--	----	---	----	--	--	--

1D Convolution

- 1D convolution example:

- Signal 'x':

0	1	1	2	3	5	8	13
---	---	---	---	---	---	---	----

- Filter 'w':

0	-1	2	-1	0
---	----	---	----	---

- Convolution 'z':

		-1	0	-1	-1		
--	--	----	---	----	----	--	--

1D Convolution Examples

- Examples:

- “Identity”

$$\hookrightarrow w = [0 \ 1 \ 0]$$

Let $x = [0 \ 1 \ 1 \ 2 \ 3 \ 5 \ 8 \ 13]$

$$z = [0 \ 1 \ 1 \ 2 \ 3 \ 5 \ 8 \ 13]$$

$0 \cdot x_0 + 1 \cdot x_1 + 0 \cdot x_2$ $0 \cdot x_1 + 1 \cdot x_2 + 0 \cdot x_3$

- “Translation”

$$\hookrightarrow w = [0 \ 0 \ 1]$$

$$z = [1 \ 1 \ 2 \ 3 \ 5 \ 8 \ 13 \ ?]$$

$0 \cdot x_0 + 0 \cdot x_1 + 1 \cdot x_2$

Translational

Don't do anything

1D Convolution Examples

- Examples:

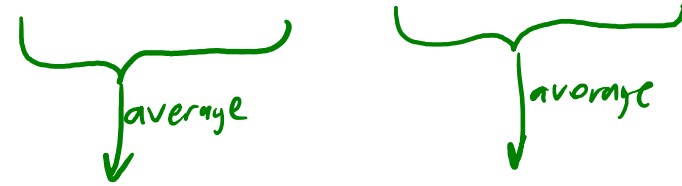
- “Identity”

$$\hookrightarrow w = [0 \ 1 \ 0]$$

- “Local Average”

$$\hookrightarrow w = \left[\frac{1}{3} \ \frac{1}{3} \ \frac{1}{3} \right]$$

$$\text{Let } x = [0 \ 1 \ 1 \ 2 \ 3 \ 5 \ 8 \ 13]$$



$$z = [? \ 2/3 \ 1/3 \ 2 \ 3\frac{1}{3} \ 5\frac{1}{3} \ 8\frac{2}{3} \ ?]$$

Boundary Issue

- What can we do about the “?” at the edges?

If $x = [0 \ 1 \ 1 \ 2 \ 3 \ 5 \ 8 \ 13]$ and $w = [\frac{1}{3} \ \frac{1}{3} \ \frac{1}{3}]$ then $z = [? \ \frac{2}{3} \ 1\frac{1}{3} \ 2 \ 3\frac{1}{3} \ 5\frac{1}{3} \ 8\frac{2}{3} \ ?]$

- Can assign values **past the boundaries**:

← • “Zero”: *most Common*
 $x = [0 \ 0 \ 0 \ 0 \ 1 \ 1 \ 2 \ 3 \ 5 \ 8 \ 13] \ 0 \ 0 \ 0$
most Popular !!!
 • “Replicate”: $x = [0 \ 0 \ 0 \ 0 \ 1 \ 1 \ 2 \ 3 \ 5 \ 8 \ 13] \ 13 \ 13 \ 13$
 • “Mirror”: $x = [2 \ 1 \ 1 \ 0 \ 1 \ 1 \ 2 \ 3 \ 5 \ 8 \ 13] \ 8 \ 5 \ 3$

- Or just ignore the “?” values and **return a shorter vector**:

$$z = [\frac{2}{3} \ 1\frac{1}{3} \ 2 \ 3\frac{1}{3} \ 5\frac{1}{3} \ 8\frac{2}{3}]$$

mirror

Formal Convolution Definition

- We've defined the convolution as:

$$Z_i = \sum_{j=-m}^m w_j x_{i+j}$$

- In other classes you may see it defined as:

$$Z_i = \sum_{j=-m}^m w_j x_{i-j}$$

(reverses 'w')

$$Z_i = \int_{-\infty}^{\infty} w_j x_{i-j} dj$$

(assumes signal + filter are continuous)

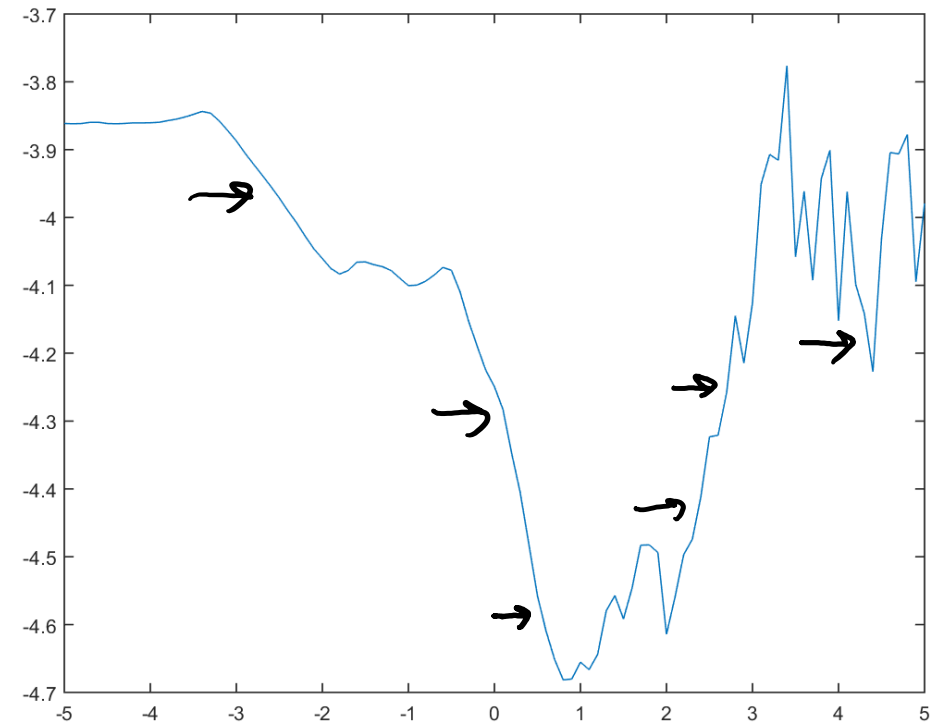
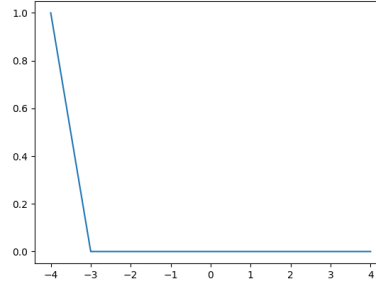
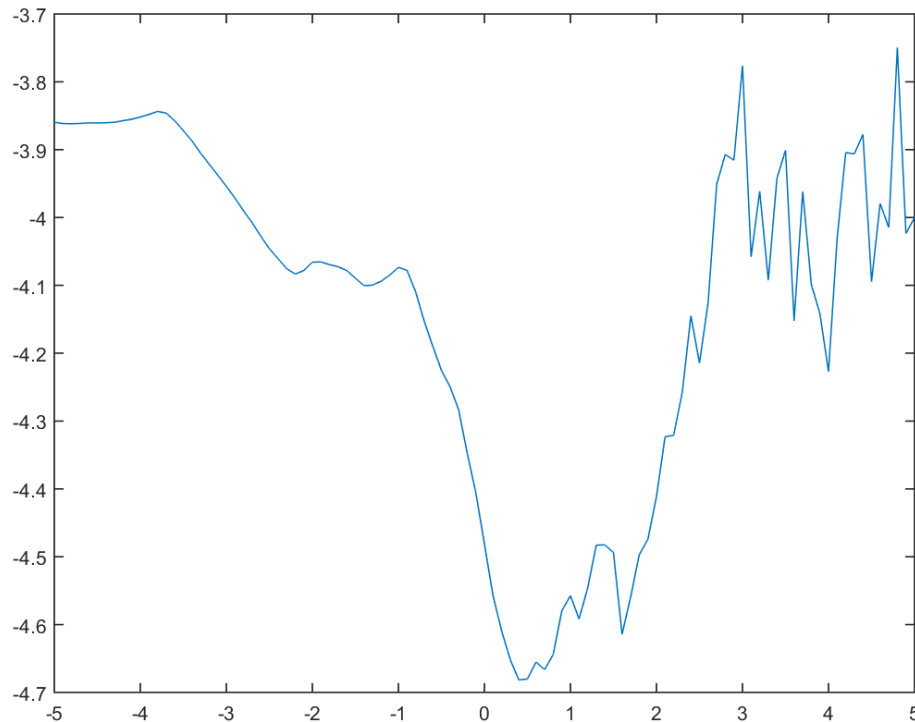
- For simplicity we use "+" instead of "-",
and assume 'w' and 'x' are sampled at discrete points (not functions).
- But keep this mind if you read about convolutions elsewhere.

Next Topic: Example Filters

1D Convolution Examples

- Translation convolution shift signal:
 - “What is my neighbour’s value?”

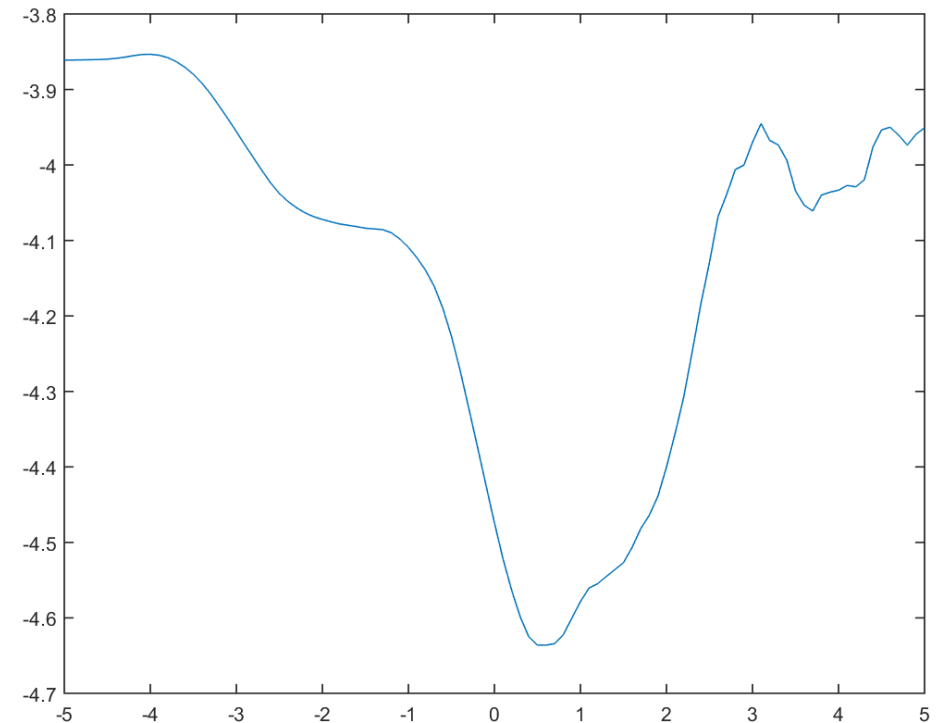
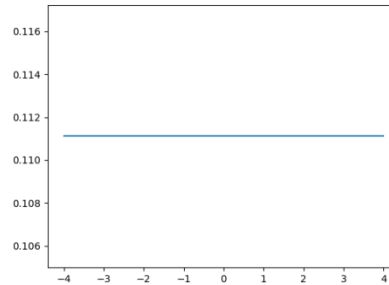
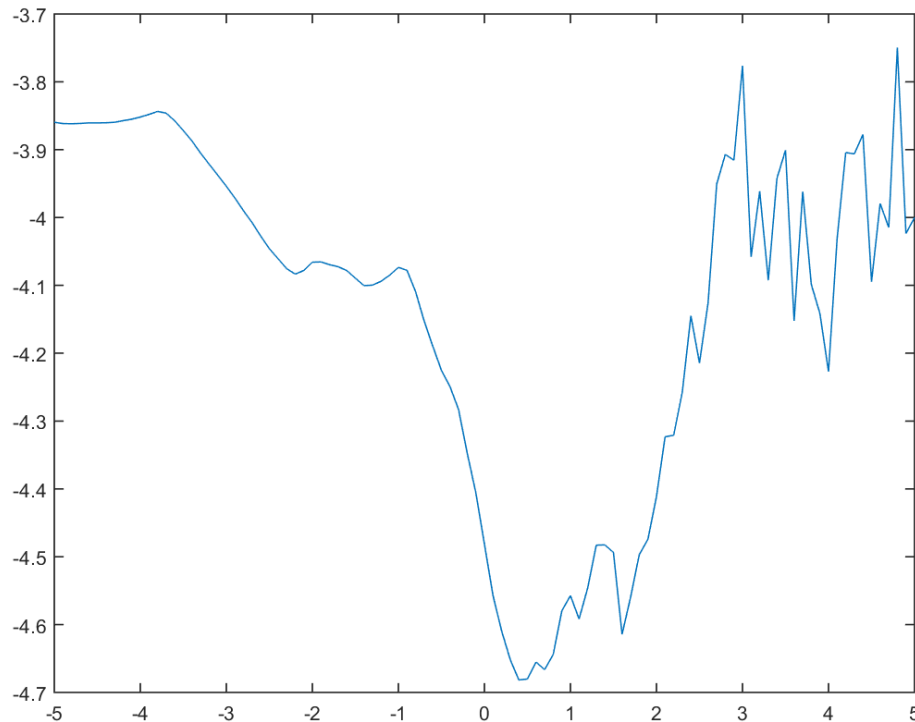
$$w = [1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0]$$



1D Convolution Examples

- **Averaging** convolution (“is signal generally high in this region?”)
 - **Less sensitive to noise** (or spikes) than raw signal.

$$w = \left[\frac{1}{9} \quad \frac{1}{9} \quad \frac{1}{9} \quad \frac{1}{9} \quad \frac{1}{9} \quad \frac{1}{9} \quad \frac{1}{9} \quad \frac{1}{9} \quad \frac{1}{9} \right]$$

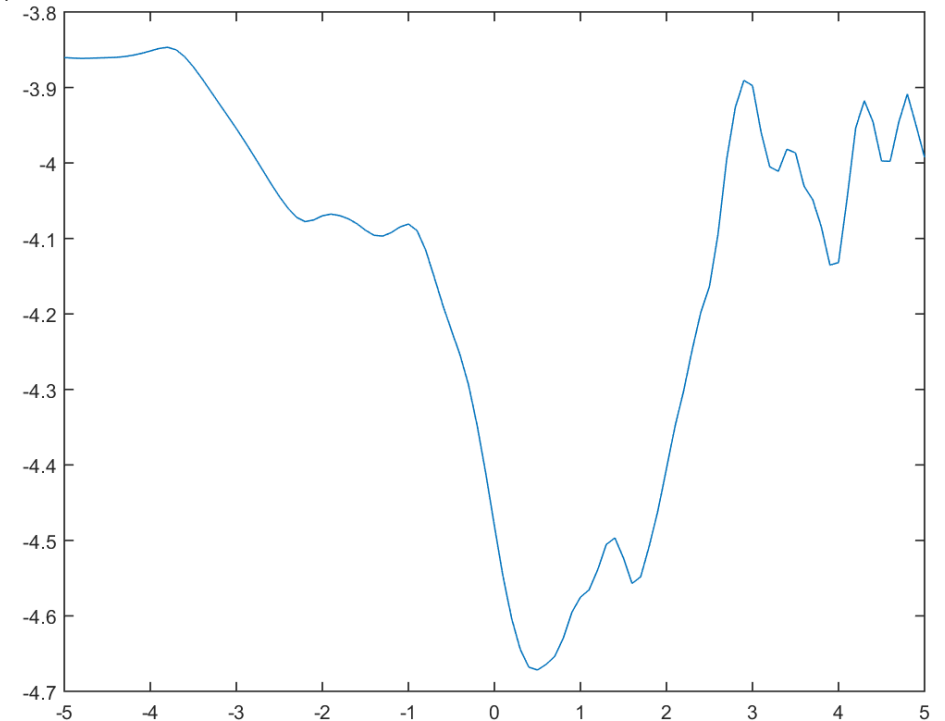
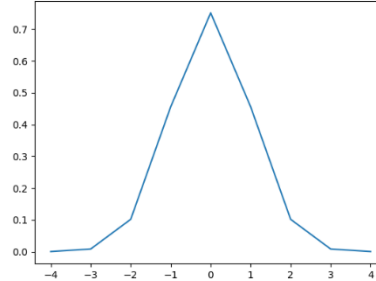
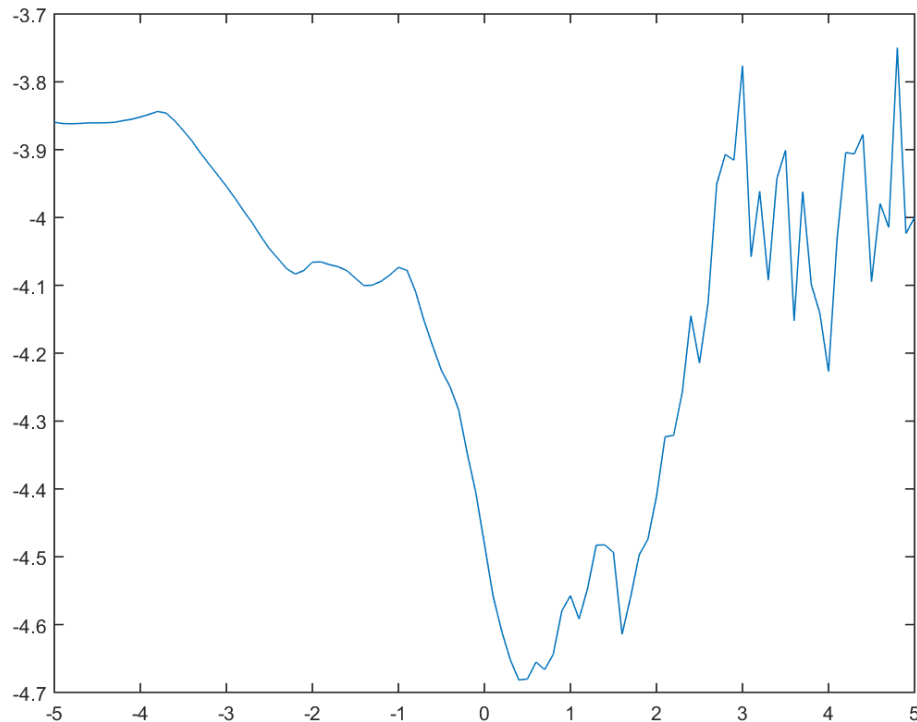


1D Convolution Examples

- **Gaussian** convolution (“blurring”): $w_i \propto \exp(-\frac{i^2}{2\sigma^2})$
 - Compared to averaging it’s more smooth and maintains peaks better.

$$W = [0.0001 \quad 0.0044 \quad 0.0540 \quad 0.2420 \quad 0.3989 \quad 0.2420 \quad 0.0540 \quad 0.0044 \quad 0.0001]$$

$(\sigma = 1, m = 4)$



1D Convolution Examples

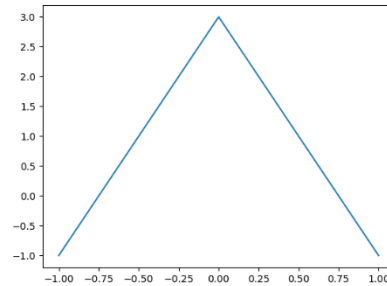
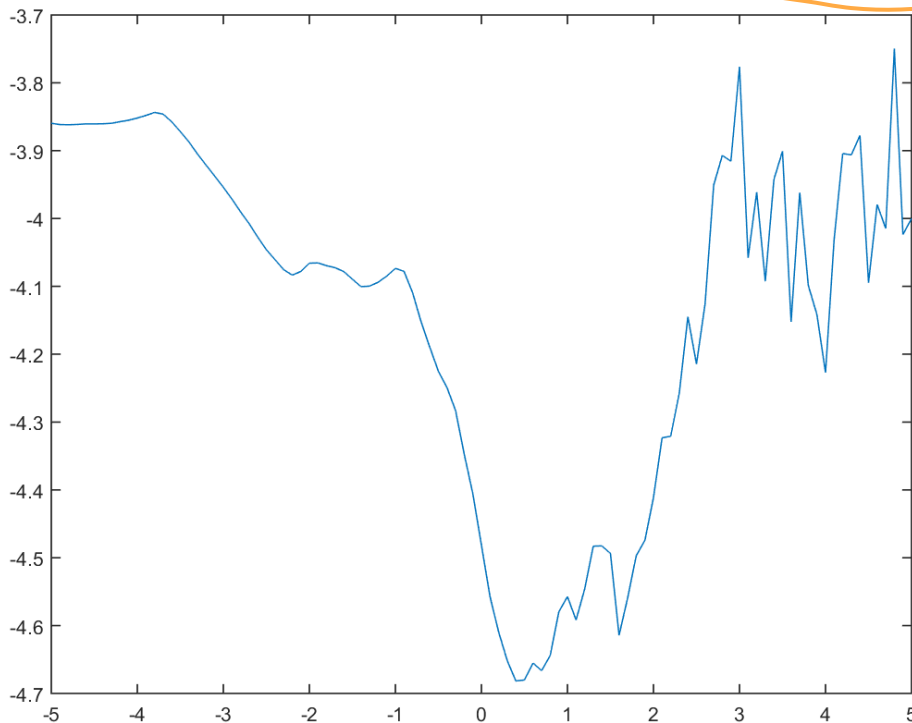
- **Sharpen** convolution enhances peaks.
 - An “average” that places **negative weights** on the surrounding pixels.

$$w = [-1 \quad 3 \quad -1]$$

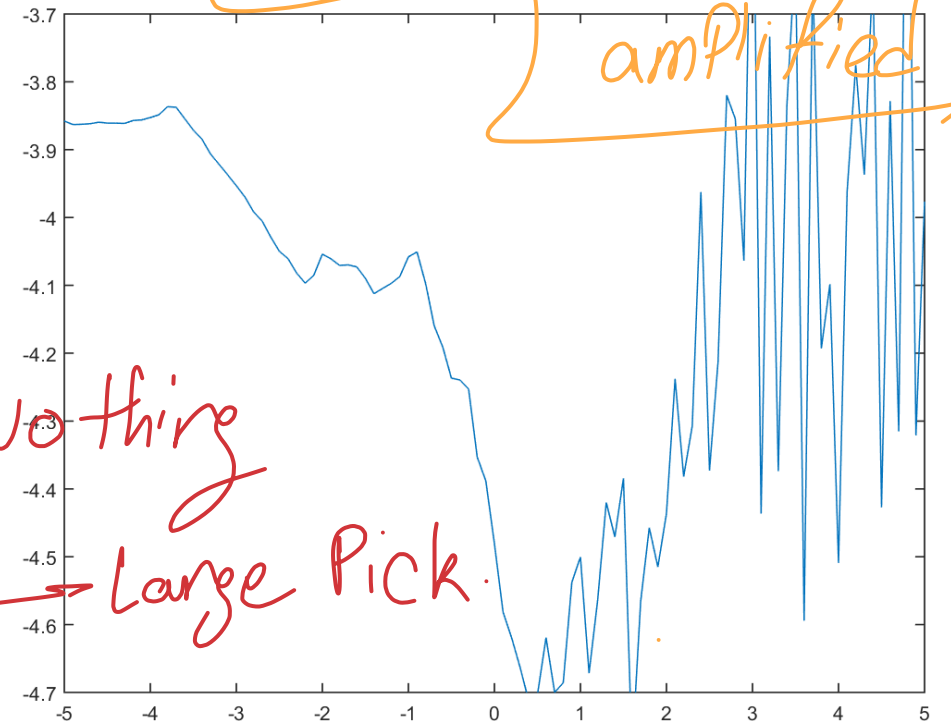
Note that this is a dot product

→ Signals similar to weights

are more amplified
→ opposite of blurring.



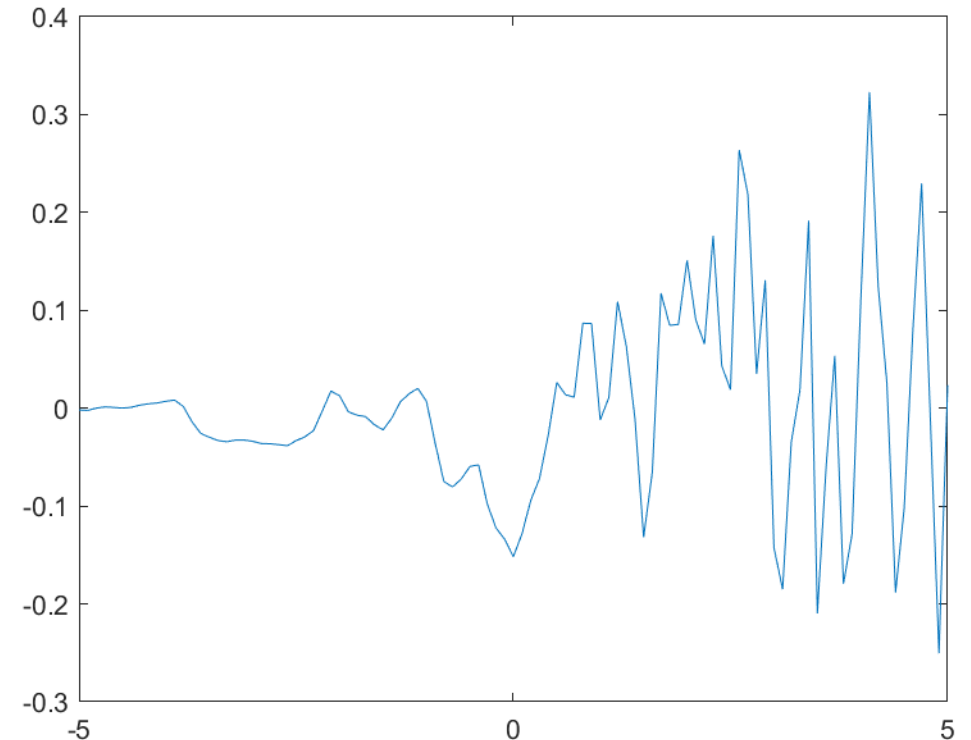
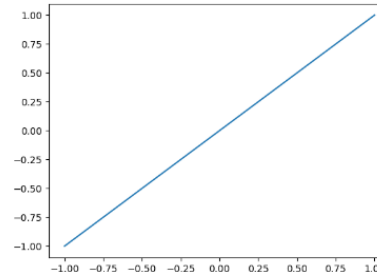
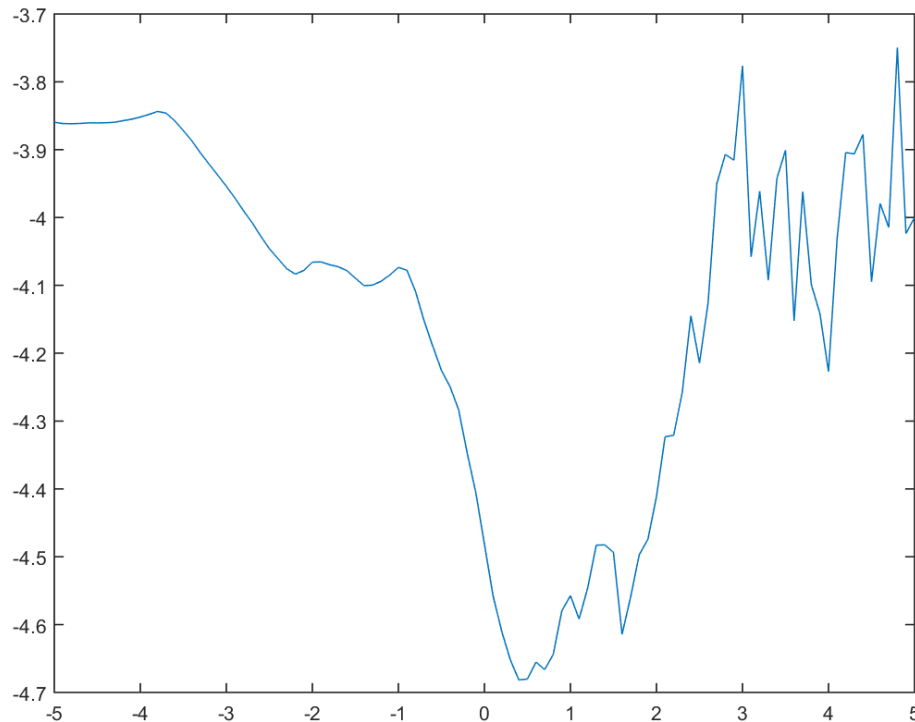
Flat line → Nothing
Small Pick → Large Pick.



1D Convolution Examples

- **Centered difference** convolution approximates **first derivative**:
 - Positive means change from low to high (negative means high to low).

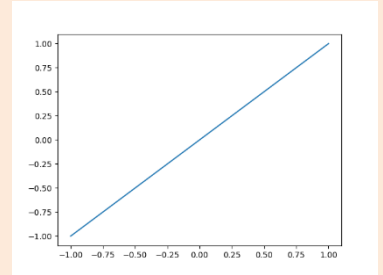
$w = [-1 \quad 0 \quad 1] \rightarrow$ Find first derivative.



Digression: Derivatives and Integrals

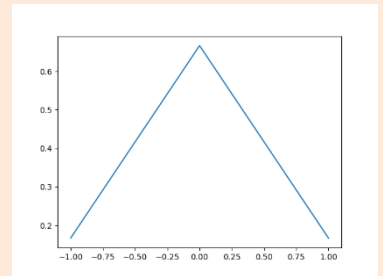
- Numerical derivative approximations can be viewed as filters:

- Centered difference: $[-1, 0, 1]$ (derivativeCheck in findMin).



- Numerical integration approximations can be viewed as filters:

- “Simpson’s” rule: $[1/6, 4/6, 1/6]$ (a bit like Gaussian filter).

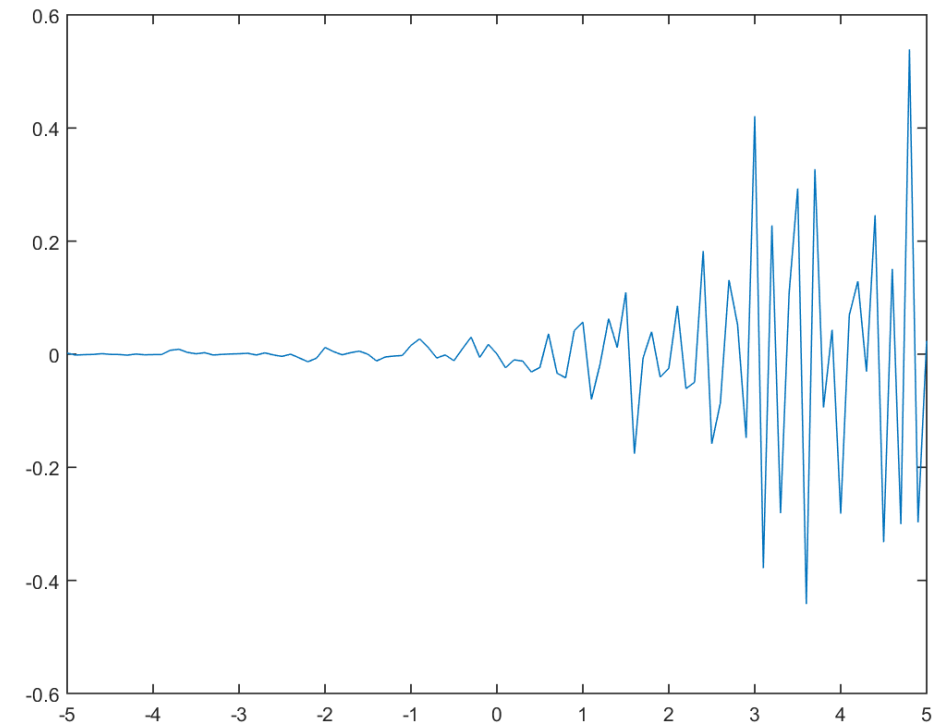
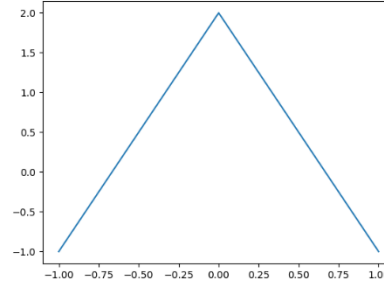
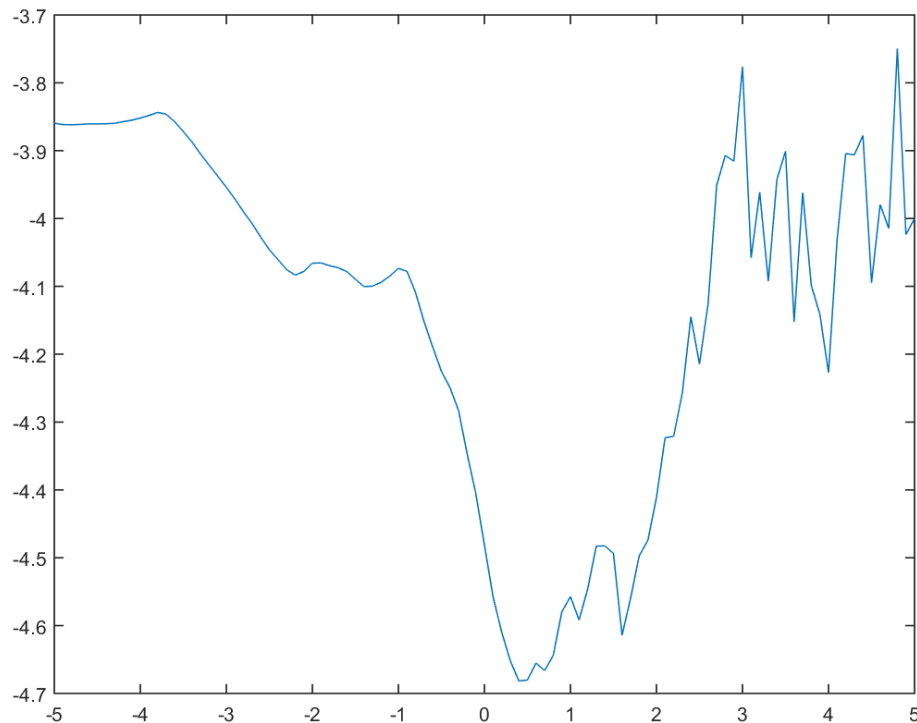


- Derivative filters add to 0, integration filters add to 1,
 - For constant function, derivative should be 0 and average = constant.

1D Convolution Examples

- Laplacian convolution approximates **second derivative**:
 - “Sum to zero” filters “**respond**” if input vector looks like the filter

$$w = [-1 \quad 2 \quad -1]$$



Laplacian of Gaussian Filter

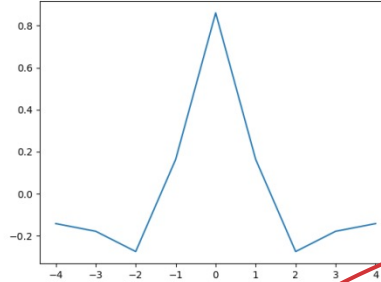
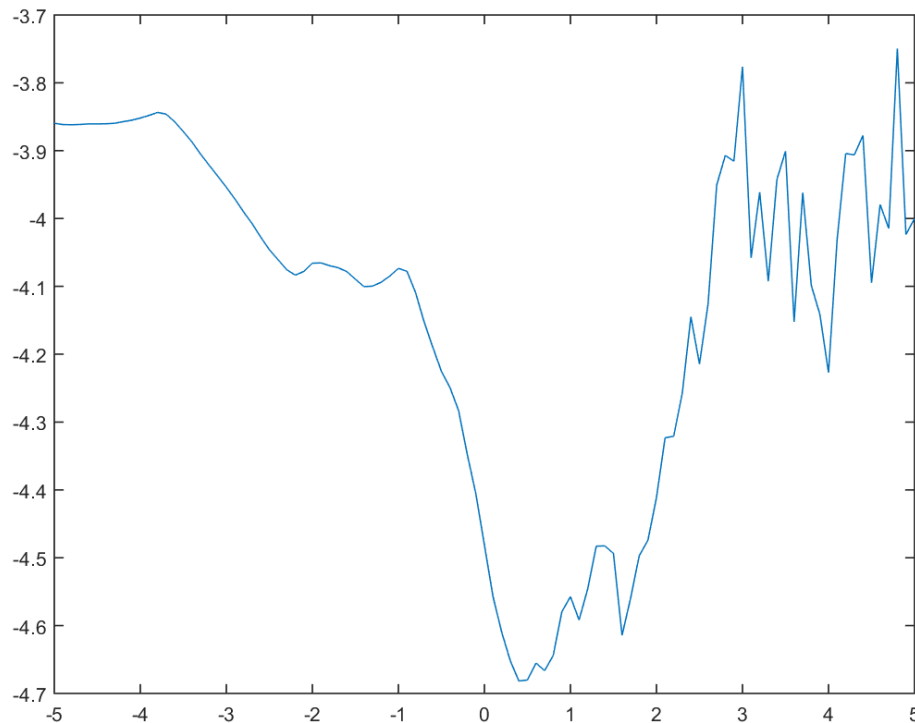
- Laplacian of Gaussian is a **smoothed 2nd-derivative** approximation:

$$w_i = \left(1 - \frac{i^2}{2\sigma^2}\right) \exp\left(-\frac{i^2}{2\sigma^2}\right)$$

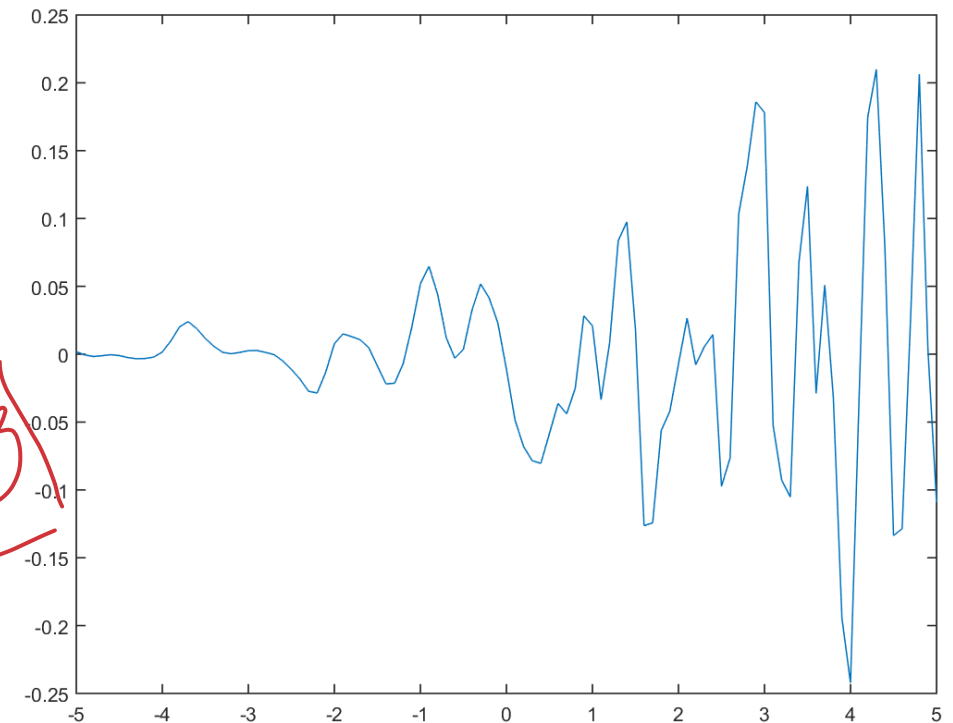
(then subtract mean)

$$w = [-0.1416 \ -0.1781 \ -0.2746 \ 0.1640 \ 0.8607 \ 0.1640 \ -0.2746 \ -0.1781 \ -0.1416]$$

$$(\sigma^2=1, m=4)$$



Signal Processing
Now



Next Topic: Convolution for Images

Images and Higher-Order Convolution

- **2D convolution:**
 - Signal 'x' is the pixel intensities in an 'n' by 'n' image.
 - Filter 'w' is the pixel intensities in a '2m+1' by '2m+1' image.
- The **2D convolution** is given by:

$$z[i_1, i_2] = \sum_{j_1=-m}^m \sum_{j_2=-m}^m w[j_1, j_2] x[i_1 + j_1, i_2 + j_2]$$

- **3D and higher-order convolutions** are defined similarly.

$$z[i_1, i_2, i_3] = \sum_{j_1=-m}^m \sum_{j_2=-m}^m \sum_{j_3=-m}^m w[j_1, j_2, j_3] x[i_1 + j_1, i_2 + j_2, i_3 + j_3]$$

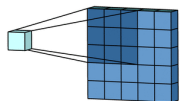
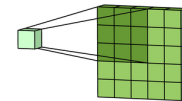
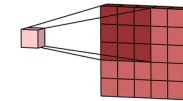
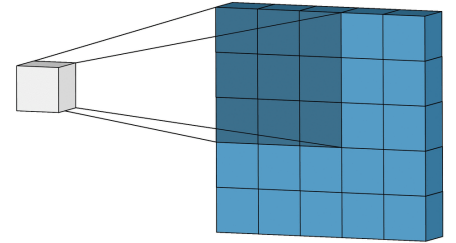
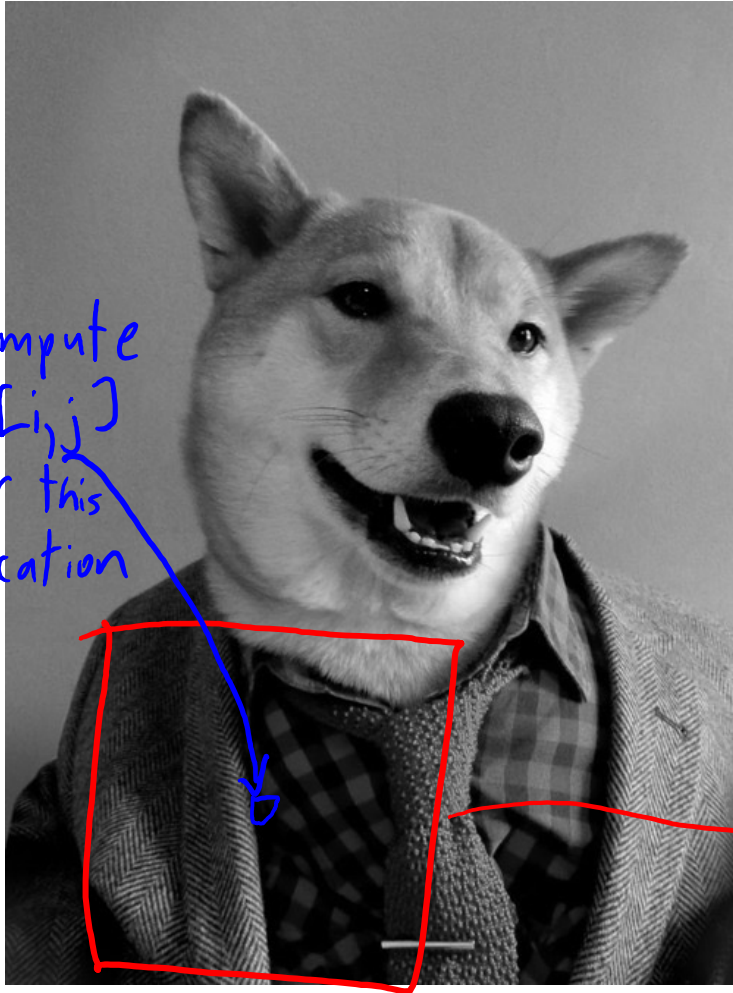


Image Convolution Examples

x

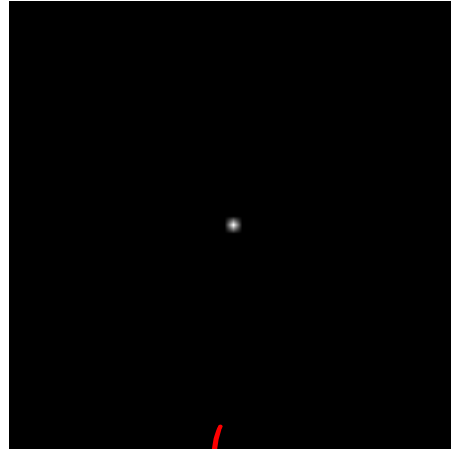


Compute $z[i,j]$ for this location

Identity convolution:
(zeros with a '1' at $w_{0,0}$)

w

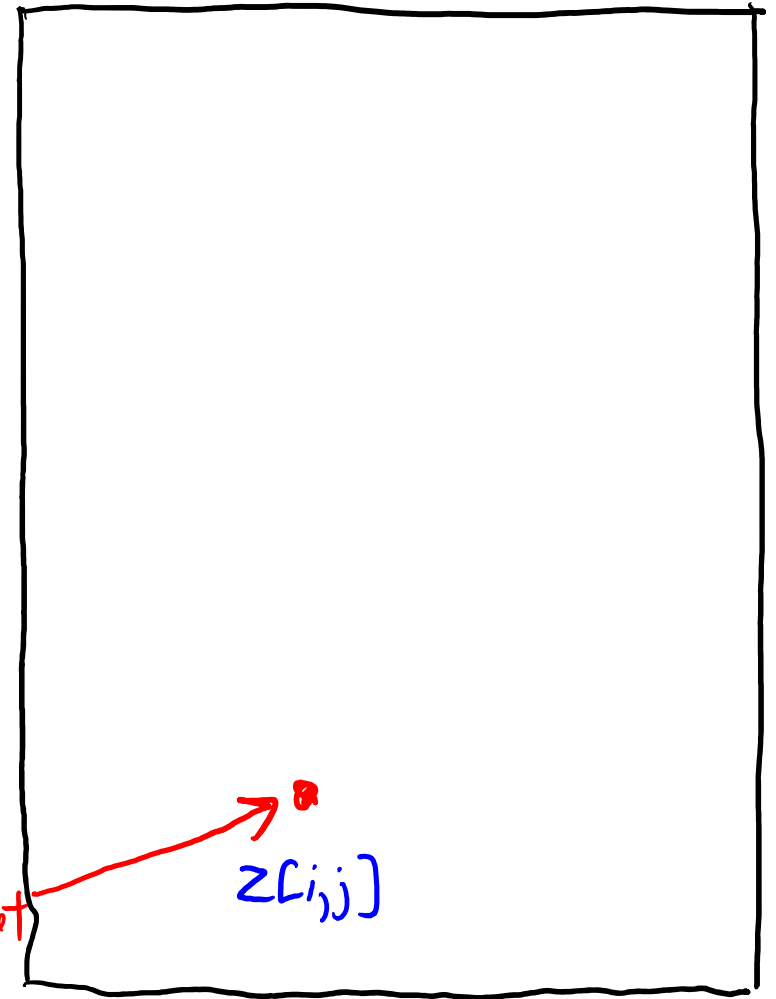
$*$



$=$

multiply element-wise
and add up result to get

z



$z[i,j]$

Image Convolution Examples

x

z

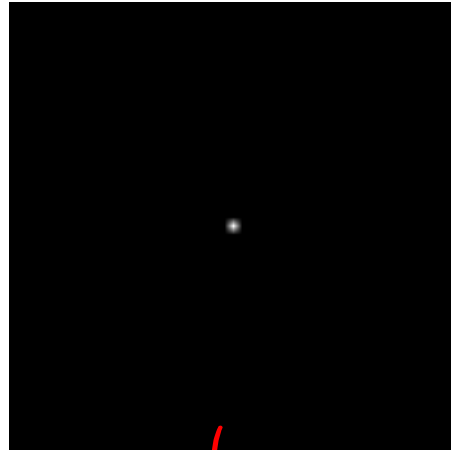
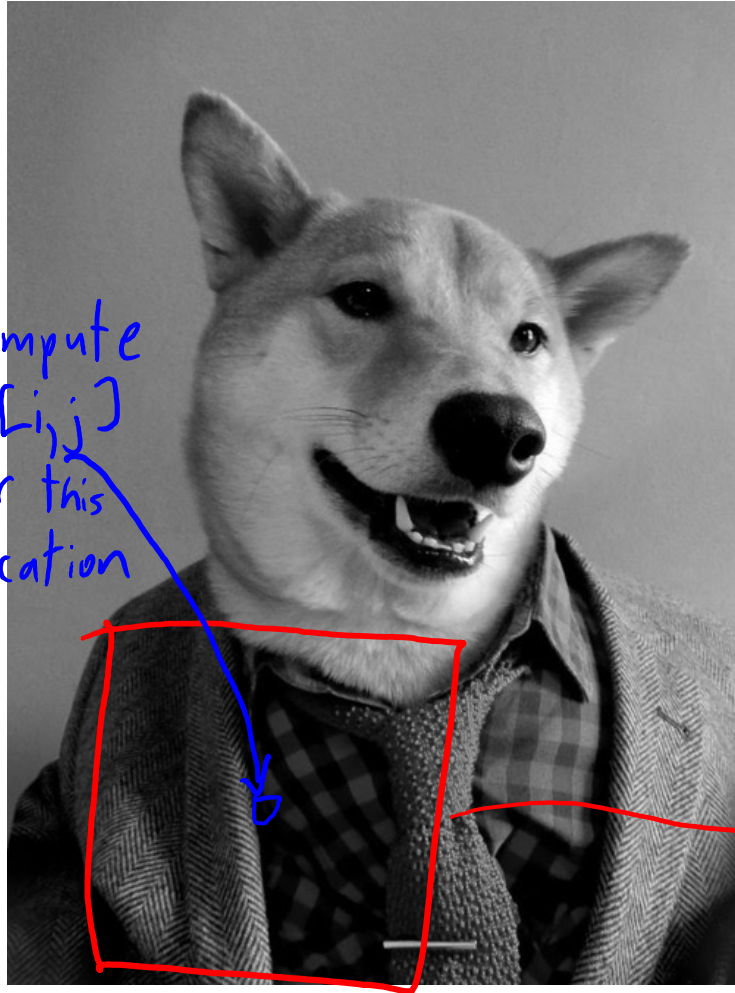
Identity convolution:
(zeros with a '1' at $w_{0,0}$)

w

$*$

$=$

Compute $z[i,j]$
for this
location



multiply element-wise
and add up result to get

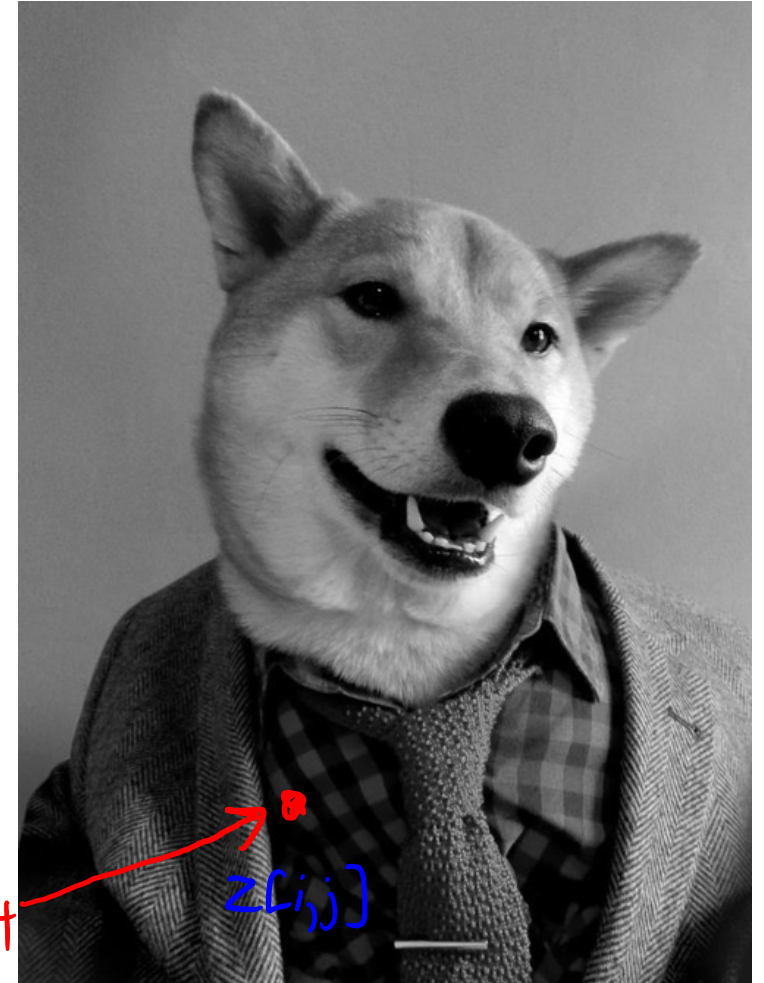


Image Convolution Examples



Translation Convolution:

$$\ast \begin{array}{c} \text{[Red circle at top-left corner]} \\ \text{[Black square]} \end{array} =$$

Boundary: "zero"



OK

Image Convolution Examples



Translation Convolution:

$$\text{Input Image} * \text{Kernel} = \text{Output Image}$$

The diagram shows a black square representing a kernel. A small red circle is drawn at the top-left corner of the square, indicating the starting point of the convolution operation.

Boundary: "replicate"



repeats

repeats

Image Convolution Examples



Translation Convolution:

$$\text{Input Image} * \text{Kernel} = \text{Output Image}$$

The diagram shows a black square representing a kernel. A red circle is drawn at the top-left corner of the square, indicating a specific point of interest or a boundary condition.

Boundary: "mirror"



flips

Image Convolution Examples



Translation Convolution:

$$\ast \begin{array}{c} \text{[Red circle at top-left corner]} \\ \text{[Black square]} \end{array} =$$

Boundary: "ignore"



X
↳ becomes smaller!

Image Convolution Examples



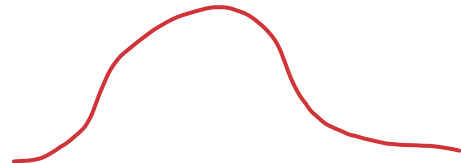
Average convolution:

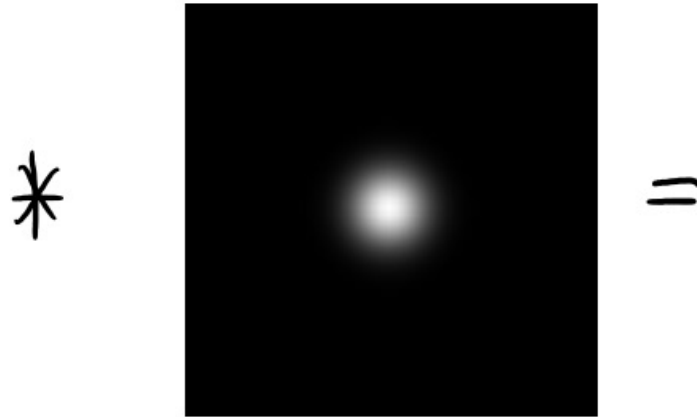
$$* \frac{1}{51} \begin{bmatrix} | & | & | & \cdots & | \\ | & | & | & \cdots & | \\ | & | & | & \cdots & | \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ | & | & | & \cdots & | \end{bmatrix} =$$



Image Convolution Examples




Gaussian Convolution:

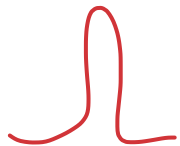


Higher variance:
blurs image to represent
average
(smoothing)



Image Convolution Examples




Gaussian Convolution:

$$* \begin{array}{c} \text{[Gaussian Kernel Image]} \\ \text{(smaller variance)} \end{array} =$$

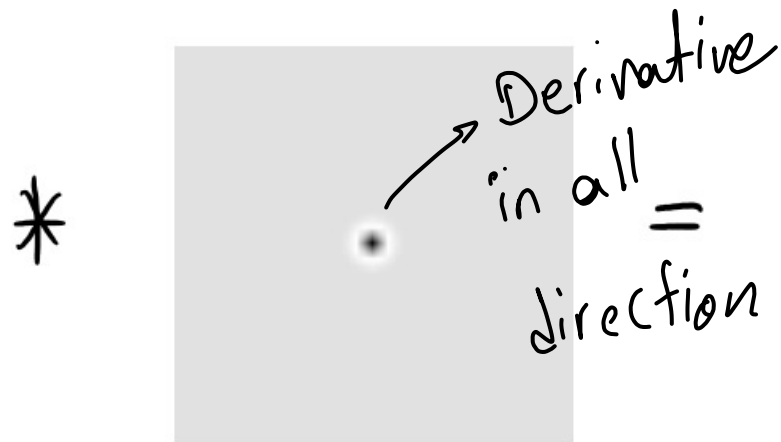
blurs image to represent
average
(smoothing)



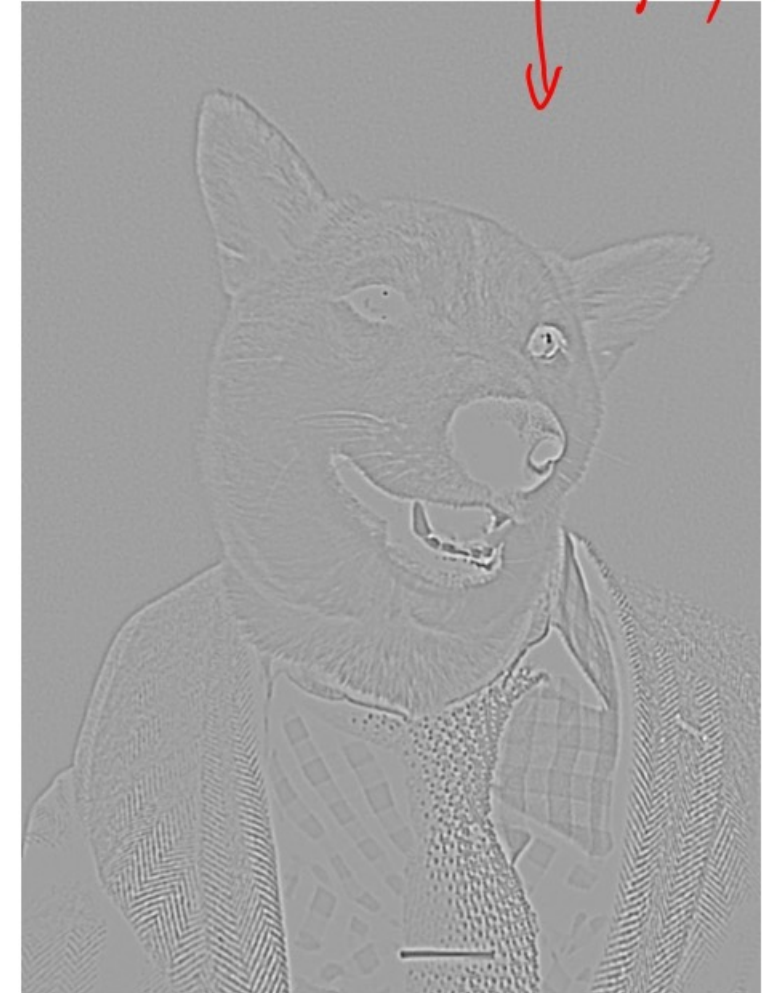
Image Convolution Examples



Laplacian of Gaussian



"How much does it look
like a black dot
surrounded by white?"



"signed"
image
(gray is 0)

This is similar to Image Convolution Examples

حالب ←

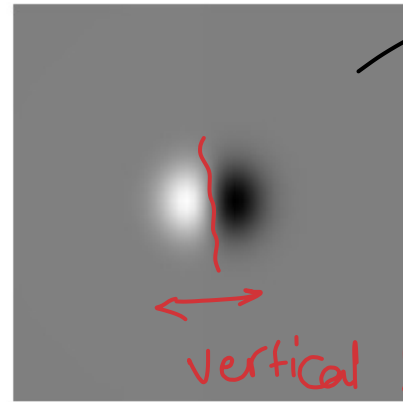
brain Cortex!

horizontal "bright to dark"



Gabor Filter
(Gaussian multiplied by
sine or cosine)

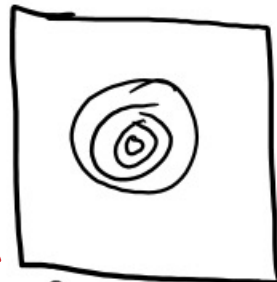
*



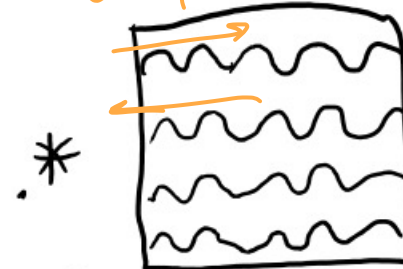
grey
is zero
=

vertical bridge (edge)

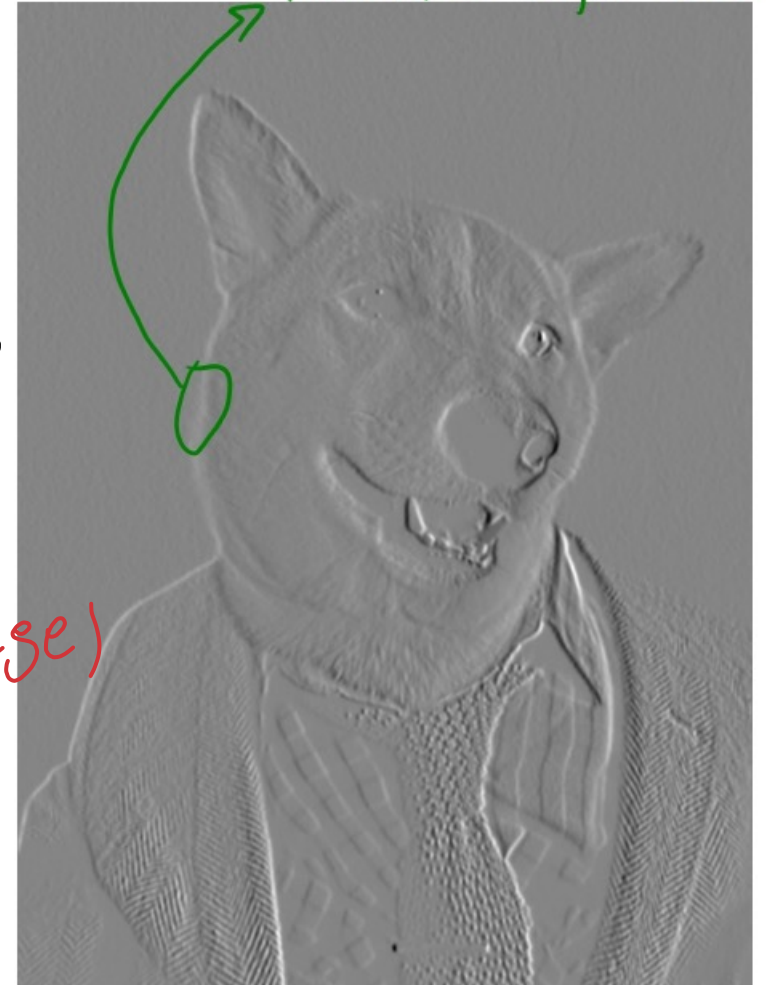
|| light { Dark



Gaussian



Parallel Sine functions



→ This is faded

↪ Since horizontal edge is not captured.

Image Convolution Examples



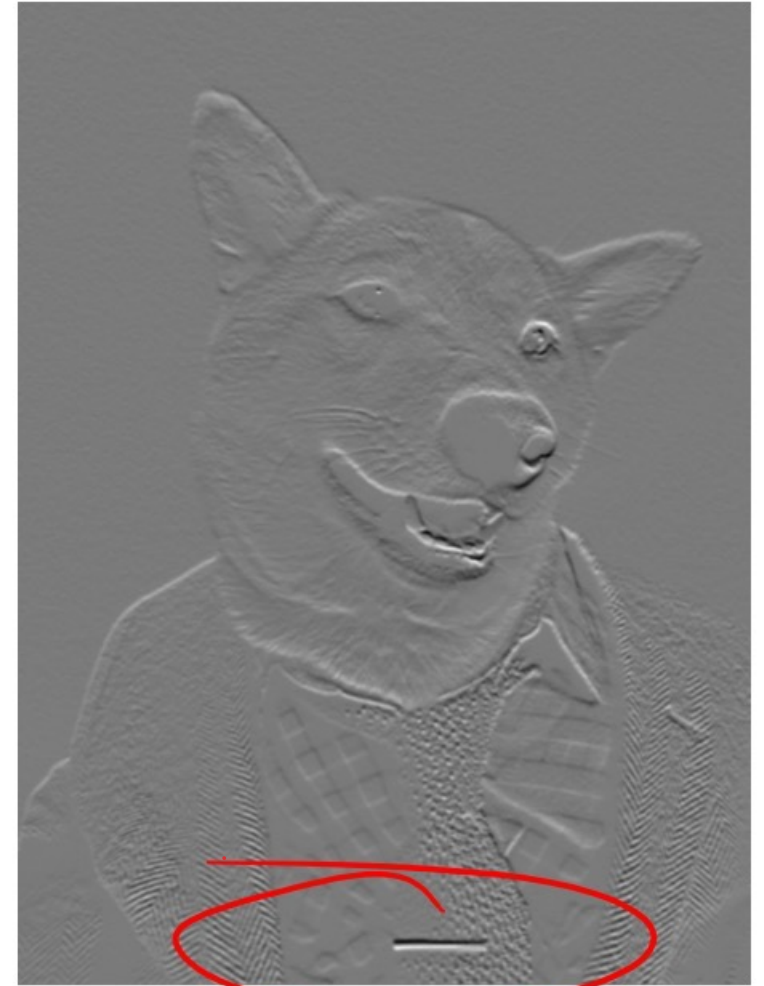
Gabor Filter
(Gaussian multiplied by
sine or cosine)

*



=

Different orientations of
the sine/cosine let us
detect changes with different
orientations.



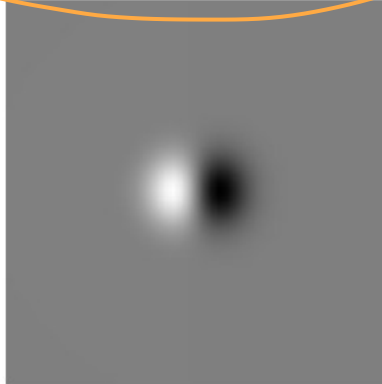
↪ 2d derivatives have a direction.

Image Convolution Examples

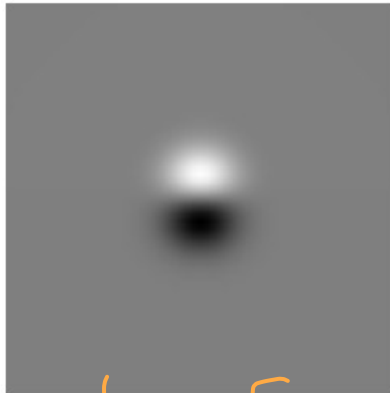


Max absolute value
between horizontal and
vertical Gabor:

*



*



حاجب
! م
.

→
maximum
absolute
value



First layer gives edge [maybe]

"Horizontal/vertical edge detector"

and then second layer gets those inputs

Summary

[you can also combine those features]

- **Convolutions** are flexible class of signal/image transformations.
 - Can approximate directional derivatives and integrals at different scales.
 - Can yield features invariant to some transformations.
- Next time:
 - Learning the convolution filters.

FFT implementation of convolution

- Convolutions can be implemented using fast Fourier transform:
 - Take FFT of image and filter, multiply elementwise, and take inverse FFT.
- It has faster asymptotic running time but there are some catches:
 - You need to be using periodic boundary conditions for the convolution.
 - Constants matter: it may not be faster in practice.
 - Especially compared to using GPUs to do the convolution in hardware.
 - The gains are largest for larger filters (compared to the image size).